Programmation Frontend

Lab 4 — Formulaires et React Router

Multi-pages, navigation et saisie de données

ESP/UCAD — 2025–2026

Dr. El Hadji Bassirou TOURÉ
Département Génie Informatique
Université Cheikh Anta Diop de Dakar

Informations pratiques

Durée estimée	2h (une séance)
Prérequis	Avoir terminé le Lab 3 (`useEffect`, loading/erreur, vue détail).
Fichiers de départ	Le projet `annuaire-react/` du Lab 3
Livrables	SPA avec 4 pages (accueil, liste, détail, ajout), navigation et formulaire
Validation	Chaque vue a sa propre URL. Le formulaire valide et envoie les données.

Objectifs du lab

À l'issue de ce lab, chaque étudiant sera capable de :

1. Installer et configurer **React Router** pour créer une application multi-pages
2. Créer des **routes**, des **liens** (`Link`) et une **barre de navigation**
3. Lire les **paramètres d'URL** avec `useParams()`
4. Créer un **formulaire contrôlé** avec `useState`
5. **Valider** les champs et afficher les erreurs
6. Envoyer des données avec `fetch` en méthode `POST`

Table des matières

1 Point de départ — Le problème du Lab 3 **3**

2 Partie 1 — Mettre en place le routeur **3**
2.1 Envelopper l'application dans `BrowserRouter` . . . 3
2.2 Créer les dossiers . . . 3
2.3 Définir les routes dans `App.jsx` . . . 4

3 Partie 2 — Navigation avec Navbar et Link **5**

4 Partie 3 — Les pages **7**
4.1 Page d'accueil 7
4.2 Page de liste 8
4.3 Modifier `UserCard` pour utiliser `Link` 10
4.4 Page de détail avec `useParams` 10

5 Partie 4 — Formulaire contrôlé **12**
5.1 Input contrôlé vs non contrôlé 12
5.2 Créer `AddUser.jsx` 13

6 Bilan — Structure du projet **16**

7 Aide-mémoire **17**

8 Résumé et conclusion **18**

1 Point de départ — Le problème du Lab 3

Au Lab 3, la navigation entre la liste et le détail est gérée par un état `selectedUserId`. Ça fonctionne, mais :

- L'**URL ne change pas**. Liste ou détail, l'adresse reste `localhost:5173`.
- Le **bouton retour** du navigateur ne fonctionne pas.
- On ne peut pas **partager un lien** vers un utilisateur précis.
- On ne peut pas **ouvrir un profil dans un nouvel onglet**.

React Router résout tout cela : chaque vue obtient sa propre URL.

Étape 0 — Installation de React Router

Dans le terminal, à la racine du projet :

```
cd annuaire-react
npm install react-router-dom
```

Vérifiez que la ligne `"react-router-dom"` apparaît dans `package.json` sous `dependencies`.

2 Partie 1 — Mettre en place le routeur

Objectif — 10 minutes

Configurer React Router et définir les routes de l'application.

2.1 Envelopper l'application dans `BrowserRouter`

Modifiez `src/main.jsx` :

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import App from "./App";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>
);
```

`BrowserRouter` active le système de routing dans toute l'application. Tous les composants enfants pourront utiliser `Link`, `useParams`, etc.

2.2 Créer les dossiers

```
mkdir -p src/pages
```

On sépare les **pages** (composants liés à une route) des **composants** réutilisables.

2.3 Définir les routes dans `App.jsx`

Remplacez **tout le contenu** de `src/App.jsx` :

```
import { Routes, Route } from "react-router-dom";
import Navbar from "./components/Navbar";
import Home from "./pages/Home";
import UserList from "./pages/UserList";
import UserDetail from "./pages/UserDetail";
import AddUser from "./pages/AddUser";
import styles from "./App.module.css";

const App = () => {
  return (
    <div className={styles.app}>
      <Navbar />
      <main className={styles.main}>
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/users"
                 element={<UserList />} />
          <Route path="/users/:id"
                 element={<UserDetail />} />
          <Route path="/users/add"
                 element={<AddUser />} />
        </Routes>
      </main>
    </div>
  );
};

export default App;
```

Les routes de l'annuaire

URL	Page	Description
/	Home	Page d'accueil
/users	UserList	Liste avec recherche, tri et favoris
/users/:id	UserDetail	Détail d'un utilisateur (:id est un paramètre)
/users/add	AddUser	Formulaire d'ajout

Remplacez `src/App.module.css` :

```
.app {
  min-height: 100vh;
  background: #f5f6fa;
}

.main {
  max-width: 700px;
  margin: 0 auto;
  padding: 2rem 1rem;
}
```

Ajoutez aussi un reset CSS global dans `index.html`, dans la balise `<head>` :

```
<style>
  *, *::before, *::after {
    box-sizing: border-box; margin: 0; padding: 0;
  }
  body {
    font-family: -apple-system, BlinkMacSystemFont,
      "Segoe UI", Roboto, sans-serif;
    color: #2d3436;
    background: #f5f6fa;
    line-height: 1.6;
  }
  a { text-decoration: none; color: inherit; }
</style>
```

App.jsx a changé de rôle

Au Lab 3, `App.jsx` contenait toute la logique (états, fetch, filtrage). Maintenant, il ne fait que **deux choses** : afficher la `Navbar` et définir les `Routes`. Chaque page gère sa propre logique. Le code est mieux organisé.

3 Partie 2 — Navigation avec Navbar et Link

Objectif — 10 minutes

Créer une barre de navigation qui fonctionne sans rechargement de page.

Créez `src/components/Navbar.jsx` :

```
import { NavLink } from "react-router-dom";
import styles from "./Navbar.module.css";

const Navbar = () => {
  return (
    <nav className={styles.nav}>
      <div className={styles.container}>
        <NavLink to="/" className={styles.logo}>
          Annuaire
        </NavLink>
        <div className={styles.links}>
          <NavLink to="/users"
            className={({ isActive }) =>
              isActive
                ? `${styles.link} ${styles.active}`
                : styles.link
            }>
            Liste
          </NavLink>
          <NavLink to="/users/add"
            className={({ isActive }) =>
              isActive
                ? `${styles.link} ${styles.active}`
                : styles.link
            }>
            + Ajouter
          </NavLink>
```

```
        </div>
      </div>
    </nav>
  );
};

export default Navbar;
```

Link vs NavLink vs <a>

Élément	Usage
`<a href="...">`	Lien HTML classique. **Recharge** toute la page. Ne jamais utiliser pour la navigation interne.
`<Link to="...">`	Lien React Router. Change l'URL **sans** recharger la page.
`<NavLink to="...">`	Comme `Link`, mais fournit en plus `isActive` pour styliser le lien de la page courante.

Créez `src/components/Navbar.module.css` :

```
.nav {
  background: #2d3436;
  padding: 0 1rem;
  position: sticky;
  top: 0;
  z-index: 10;
}

.container {
  max-width: 700px;
  margin: 0 auto;
  display: flex;
  justify-content: space-between;
  align-items: center;
  height: 56px;
}

.logo {
  font-size: 1.25rem;
  font-weight: 700;
  color: #fff;
}

.links { display: flex; gap: 0.25rem; }

.link {
  color: #b2bec3;
  padding: 0.4rem 0.9rem;
  border-radius: 6px;
  font-size: 0.9rem;
  transition: background 0.15s, color 0.15s;
}

.link:hover {
  color: #fff;
```

```
  background: rgba(255, 255, 255, 0.1);
}

.active {
  color: #fff;
  background: rgba(255, 255, 255, 0.15);
}
```

4 Partie 3 — Les pages

Objectif — 15 minutes

Créer les pages Home, UserList et déplacer UserDetail.

4.1 Page d'accueil

Créez `src/pages/Home.jsx` :

```
import { Link } from "react-router-dom";
import styles from "./Home.module.css";

const Home = () => {
  return (
    <div className={styles.home}>
      <h1 className={styles.titre}>
        Annuaire des utilisateurs
      </h1>
      <p className={styles.description}>
        Consultez, recherchez et ajoutez des utilisateurs.
      </p>
      <div className={styles.actions}>
        <Link to="/users" className={styles.bouton}>
          Voir la liste
        </Link>
        <Link to="/users/add"
              className={styles.boutonSecondaire}>
          Ajouter un utilisateur
        </Link>
      </div>
    </div>
  );
};

export default Home;
```

Créez `src/pages/Home.module.css` :

```
.home { text-align: center; padding: 3rem 1rem; }

.titre {
  font-size: 2rem; color: #2d3436;
  margin-bottom: 1rem;
}

.description {
  color: #636e72; font-size: 1.05rem;
```

```
  max-width: 450px; margin: 0 auto 2rem auto;
}

.actions {
  display: flex; gap: 1rem;
  justify-content: center; flex-wrap: wrap;
}

.bouton {
  padding: 0.7rem 1.8rem; background: #0984e3;
  color: #fff; border-radius: 8px;
  font-weight: 500; transition: background 0.15s;
}
.bouton:hover { background: #0770c2; }

.boutonSecondaire {
  padding: 0.7rem 1.8rem; background: #fff;
  color: #0984e3; border: 1px solid #0984e3;
  border-radius: 8px; font-weight: 500;
}
.boutonSecondaire:hover { background: #ebf5fb; }
```

4.2 Page de liste

Créez `src/pages/UserList.jsx`. Cette page reprend la logique qui était dans `App.jsx` au Lab 3 :

```
import { useState, useEffect } from "react";
import UserCard from "../components/UserCard";
import SearchBar from "../components/SearchBar";
import styles from "./UserList.module.css";

const UserList = () => {
  const [users, setUsers] = useState([]);
  const [chargement, setChargement] = useState(true);
  const [erreur, setErreur] = useState(null);
  const [recherche, setRecherche] = useState("");
  const [triAscendant, setTriAscendant] = useState(true);
  const [favoris, setFavoris] = useState([]);

  useEffect(() => {
    const charger = async () => {
      try {
        const response = await fetch(
          "https://jsonplaceholder.typicode.com/users"
        );
        if (!response.ok)
          throw new Error("Erreur serveur");
        const data = await response.json();
        setUsers(data);
      } catch (error) {
        setErreur(error.message);
      }
      setChargement(false);
    };
    charger();
  }, []);
```

```
  const toggleFavori = (userId) => {
    if (favoris.includes(userId)) {
      setFavoris(
        favoris.filter((id) => id !== userId)
      );
    } else {
      setFavoris([...favoris, userId]);
    }
  };

  if (chargement)
    return <p>Chargement en cours...</p>;
  if (erreur)
    return <p className={styles.erreur}>
      Erreur : {erreur}</p>;

  const usersFiltres = users
    .filter((u) =>
      u.name.toLowerCase()
        .includes(recherche.toLowerCase())
    )
    .sort((a, b) =>
      triAscendant
        ? a.name.localeCompare(b.name)
        : b.name.localeCompare(a.name)
    );

  return (
    <div>
      <h1 className={styles.titre}>
        Liste des utilisateurs
      </h1>
      <SearchBar recherche={recherche}
                 onRecherche={setRecherche} />
      <div className={styles.barre}>
        <span className={styles.compteur}>
          {usersFiltres.length} utilisateur(s)
        </span>
        {favoris.length > 0 && (
          <span className={styles.favoris}>
            {favoris.length} favori(s)
          </span>
        )}
        <button className={styles.boutonTri}
                onClick={() =>
                  setTriAscendant(!triAscendant)
                }>
          {triAscendant ? "A → Z" : "Z → A"}
        </button>
      </div>
      {usersFiltres.map((user) => (
        <UserCard key={user.id} user={user}
          estFavori={favoris.includes(user.id)}
          onToggleFavori={toggleFavori} />
      ))}
    </div>
  );
```

```
};

export default UserList;
```

4.3 Modifier `UserCard` pour utiliser `Link`

Au Lab 3, les cartes utilisaient `onClick` pour naviguer par état. Maintenant, on utilise `Link` pour naviguer par URL. Modifiez `src/components/UserCard.jsx` :

```
import { Link } from "react-router-dom";
import styles from "./UserCard.module.css";

const UserCard = ({
  user, estFavori, onToggleFavori
}) => {
  return (
    <div className={styles.card}>
      <div className={styles.header}>
        <Link to={`/users/${user.id}`}
              className={styles.lien}>
          <h3 className={styles.nom}>{user.name}</h3>
        </Link>
        <button className={styles.etoile}
                onClick={() =>
                  onToggleFavori(user.id)
                }>
          {estFavori ? "\u2605" : "\u2606"}
        </button>
      </div>
      <p className={styles.info}>
        Email : {user.email}
      </p>
      <p className={styles.info}>
        Entreprise : {user.company.name}
      </p>
      <Link to={`/users/${user.id}`}
            className={styles.voir}>
        Voir le profil →
      </Link>
    </div>
  );
};

export default UserCard;
```

Plus besoin de `e.stopPropagation()`

Au Lab 3, le clic sur l'étoile déclenchait aussi le `onClick` de la carte. On avait besoin de `e.stopPropagation()`. Maintenant, la navigation se fait par `Link` (un lien, pas un `onClick` sur la carte entière). Le bouton étoile n'entre plus en conflit.

4.4 Page de détail avec `useParams`

Créez `src/pages/UserDetail.jsx`. Le composant lit l'identifiant **depuis l'URL** au lieu de le recevoir en prop :

```
import { useState, useEffect } from "react";
import { useParams, Link } from "react-router-dom";
import styles from "./UserDetail.module.css";

const UserDetail = () => {
  const { id } = useParams();
  const [user, setUser] = useState(null);
  const [chargement, setChargement] = useState(true);
  const [erreur, setErreur] = useState(null);

  useEffect(() => {
    const charger = async () => {
      setChargement(true);
      try {
        const response = await fetch(
          `https://jsonplaceholder.typicode.com/users/${id}`
        );
        if (!response.ok)
          throw new Error("Utilisateur introuvable");
        const data = await response.json();
        setUser(data);
      } catch (error) {
        setErreur(error.message);
      }
      setChargement(false);
    };
    charger();
  }, [id]);

  if (chargement) return <p>Chargement...</p>;

  if (erreur) {
    return (
      <div>
        <p>{erreur}</p>
        <Link to="/users">Retour à la liste</Link>
      </div>
    );
  }

  return (
    <div>
      <Link to="/users" className={styles.retour}>←
         Retour à la liste
      </Link>
      <div className={styles.profil}>
        <div className={styles.avatar}>
          {user.name.charAt(0)}
        </div>
        <h1 className={styles.nom}>{user.name}</h1>
        <p className={styles.username}>
          @{user.username}
        </p>
      </div>
      <div className={styles.carte}>
        <h2 className={styles.section}>Coordonnées</h2>
        <div className={styles.champ}>
```

```
          <span className={styles.label}>Email</span>
          <span>{user.email}</span>
        </div>
        <div className={styles.champ}>
          <span className={styles.label}>Tél.</span>
          <span>{user.phone}</span>
        </div>
        <div className={styles.champ}>
          <span className={styles.label}>Site</span>
          <span>{user.website}</span>
        </div>
      </div>
      <div className={styles.carte}>
        <h2 className={styles.section}>Entreprise</h2>
        <div className={styles.champ}>
          <span className={styles.label}>Nom</span>
          <span>{user.company.name}</span>
        </div>
        <div className={styles.champ}>
          <span className={styles.label}>Ville</span>
          <span>{user.address.city}</span>
        </div>
      </div>
    </div>
  );
};

export default UserDetail;
```

Lab 3 vs Lab 4 : la page de détail

	Lab 3	**Lab 4**
L'identifiant vient de	Une prop `userId` (passée par App)	L'URL via `useParams()`
L'URL	Ne change pas	`/users/3`
Bouton retour navigateur	Ne fonctionne pas	Fonctionne
Partage de lien	Impossible	Possible

Supprimez l'ancien `src/components/UserDetail.jsx` et son CSS Module — il est remplacé par la page. Les fichiers CSS des pages (`UserDetail.module.css`, `UserList.module.css`) sont fournis dans le script de setup.

5 Partie 4 — Formulaire contrôlé

Objectif — 20 minutes

Créer un formulaire pour ajouter un utilisateur. Comprendre les inputs contrôlés.

5.1 Input contrôlé vs non contrôlé

Un **input contrôlé** est un champ dont la valeur est stockée dans l'état React. À chaque frappe, l'état se met à jour, et React re-rend le composant avec la nouvelle valeur.

	Non contrôlé	Contrôlé
Valeur stockée dans	Le DOM (le navigateur)	L'état React
Lecture	`inputRef.current.value`	`nom` (variable d'état)
Avantage	Simple pour les cas basiques	Total contrôle (validation, formatage)

En React, on utilise **toujours** des inputs contrôlés. Le principe : `value={état}` + `onChange={fn}`.

5.2 Créer `AddUser.jsx`

Créez `src/pages/AddUser.jsx` :

```
import { useState } from "react";
import { useNavigate } from "react-router-dom";
import styles from "./AddUser.module.css";

const AddUser = () => {
  const navigate = useNavigate();

  const [form, setForm] = useState({
    name: "",
    email: "",
    phone: "",
    company: "",
  });
  const [erreurs, setErreurs] = useState({});
  const [enCours, setEnCours] = useState(false);
  const [soumis, setSoumis] = useState(false);

  const handleChange = (e) => {
    const { name, value } = e.target;
    setForm({ ...form, [name]: value });
    if (erreurs[name]) {
      setErreurs({ ...erreurs, [name]: null });
    }
  };

  const valider = () => {
    const err = {};
    if (!form.name.trim())
      err.name = "Le nom est obligatoire";
    if (!form.email.trim())
      err.email = "L'email est obligatoire";
    else if (!form.email.includes("@"))
      err.email = "L'email doit contenir un @";
    if (!form.phone.trim())
      err.phone = "Le téléphone est obligatoire";
    return err;
  };

  const handleSubmit = async (e) => {
    e.preventDefault();

    const err = valider();
    if (Object.keys(err).length > 0) {
      setErreurs(err);
      return;
```

```
    }

    setEnCours(true);
    try {
      const response = await fetch(
        "https://jsonplaceholder.typicode.com/users",
        {
          method: "POST",
          headers: {
            "Content-Type": "application/json",
          },
          body: JSON.stringify({
            name: form.name,
            email: form.email,
            phone: form.phone,
            company: {
              name: form.company || "Non renseigné",
            },
          }),
        }
      );
      if (!response.ok)
        throw new Error("Erreur serveur");

      const data = await response.json();
      console.log("Utilisateur créé :", data);
      setSoumis(true);
      setTimeout(() => navigate("/users"), 2000);
    } catch (error) {
      setErreurs({ submit: error.message });
    }
    setEnCours(false);
  };

  if (soumis) {
    return (
      <div className={styles.succes}>
        <h2>Utilisateur ajouté</h2>
        <p>Redirection vers la liste...</p>
      </div>
    );
  }

  return (
    <div>
      <h1 className={styles.titre}>
        Ajouter un utilisateur
      </h1>
      <div className={styles.carte}>
        <form onSubmit={handleSubmit}>
          <div className={styles.groupe}>
            <label className={styles.label}
                   htmlFor="name">
              Nom complet *
            </label>
            <input className={styles.input}
              type="text" id="name" name="name"
              placeholder="Fatou Ndiaye"
```

```
            value={form.name}
            onChange={handleChange} />
          {erreurs.name && (
            <p className={styles.erreur}>
              {erreurs.name}
            </p>
          )}
        </div>

        <div className={styles.groupe}>
          <label className={styles.label}
                 htmlFor="email">
            Email *
          </label>
          <input className={styles.input}
            type="text" id="email" name="email"
            placeholder="fatou.ndiaye@ucad.edu.sn"
            value={form.email}
            onChange={handleChange} />
          {erreurs.email && (
            <p className={styles.erreur}>
              {erreurs.email}
            </p>
          )}
        </div>

        <div className={styles.groupe}>
          <label className={styles.label}
                 htmlFor="phone">
            Téléphone *
          </label>
          <input className={styles.input}
            type="text" id="phone" name="phone"
            placeholder="77 123 45 67"
            value={form.phone}
            onChange={handleChange} />
          {erreurs.phone && (
            <p className={styles.erreur}>
              {erreurs.phone}
            </p>
          )}
        </div>

        <div className={styles.groupe}>
          <label className={styles.label}
                 htmlFor="company">
            Entreprise
          </label>
          <input className={styles.input}
            type="text" id="company" name="company"
            placeholder="ESP/UCAD"
            value={form.company}
            onChange={handleChange} />
        </div>

        {erreurs.submit && (
          <p className={styles.erreurGlobale}>
            {erreurs.submit}
```

```
            </p>
          )}

          <button type="submit"
                  className={styles.bouton}
                  disabled={enCours}>
            {enCours ? "Envoi..." : "Ajouter"}
          </button>
        </form>
      </div>
    </div>
  );
};

export default AddUser;
```

Décoder le formulaire

Élément	Rôle
`value={form.name}`	L'input affiche la valeur de l'état (input contrôlé)
`onChange={handleChange}`	À chaque frappe, met à jour l'état
`e.preventDefault()`	Empêche le rechargement de la page à la soumission
`name="name"`	L'attribut `name` permet à `handleChange` de savoir quel champ mettre à jour
`[name]: value`	Syntaxe ES6 : clé dynamique dans un objet
`useNavigate()`	Permet de naviguer par code (après soumission)
`method: "POST"`	Envoie les données au serveur (au lieu de les lire)

JSONPlaceholder simule le POST

L'API JSONPlaceholder accepte les requêtes POST et retourne un objet avec un `id` généré. Mais les données ne sont **pas réellement enregistrées**. Si vous rechargez la liste, le nouvel utilisateur n'apparaît pas. C'est normal — c'est une API de test. Au Lab 6, on se connectera à un vrai backend Django.

Les fichiers CSS pour les pages (`AddUser.module.css`, `UserDetail.module.css`, `UserList.module.css`) sont fournis dans le script de setup. Ils suivent la même logique que les labs précédents : un fichier `.module.css` par composant.

6 Bilan — Structure du projet

```
annuaire-react/
  src/
    App.jsx                <-- Routes seulement
    App.module.css
    components/
      Navbar.jsx           <-- nouveau
      Navbar.module.css    <-- nouveau
      UserCard.jsx         <-- modifié (Link)
      UserCard.module.css
```

```
      SearchBar.jsx
      SearchBar.module.css
    pages/                <-- nouveau dossier
      Home.jsx
      Home.module.css
      UserList.jsx        <-- logique déplacée ici
      UserList.module.css
      UserDetail.jsx      <-- useParams
      UserDetail.module.css
      AddUser.jsx         <-- nouveau
      AddUser.module.css
  index.html
  package.json
```

Avant / après : Lab 3 vs Lab 4

	Lab 3	**Lab 4**
Navigation	État `selectedUserId`	URL (`/users`, `/users/3`)
Bouton retour	Ne fonctionne pas	Fonctionne
Partage de lien	Impossible	`/users/3`
Ajout	Aucun	Formulaire contrôlé + POST
Lien actif	Aucun	NavLink avec style actif
App.jsx	Toute la logique	Routes seulement

7 Aide-mémoire

Concept	**Syntaxe**
Installer	`npm install react-router-dom`
Envelopper	`<BrowserRouter><App /></BrowserRouter>`
Définir une route	`<Route path="/x" element={<Page />} />`
Paramètre d'URL	`path="/users/:id"`
Lire le paramètre	`const { id } = useParams();`
Lien sans rechargement	`<Link to="/users">...</Link>`
Lien avec style actif	`<NavLink to="/users" className={...}>`
Navigation par code	`const navigate = useNavigate(); navigate("/users");`
Input contrôlé	`<input value={état} onChange={fn} />`
Empêcher le rechargement	`e.preventDefault()`
Clé dynamique	`setForm({ ...form, [name]: value })`
POST avec fetch	`fetch(url, { method: "POST", body: JSON.stringify(data) })`

8 Résumé et conclusion

Dans ce lab, vous avez :

1. Installé **React Router** et défini 4 routes avec leurs URL.
2. Créé une **Navbar** avec des `NavLink` qui indiquent la page active.
3. Utilisé `useParams()` pour lire l'identifiant de l'utilisateur depuis l'URL.
4. Créé un **formulaire contrôlé** avec validation et envoi en POST.
5. Utilisé `useNavigate()` pour rediriger après la soumission.

Étape	Statut	Ce que vous savez faire
DOM et événements	✓	Sélecteurs, événements
Fonctions JS	✓	Déclaration, expression, fléchée
Asynchrone	✓	`fetch`, `async/await`
React — les bases	✓	Composant, JSX, Vite
Composants et props	✓	Découpage, props, CSS Modules
État React	✓	`useState`, re-rendu, immutabilité
useEffect	✓	Montage, dépendances, loading/erreur
Router et formulaires	✓	Routes, Link, useParams, inputs contrôlés, POST
Next.js	→ Lab 5	App Router, server vs client
API Django	→ Lab 6	CORS, auth, CRUD

Et ensuite ?

L'annuaire est une SPA complète avec 4 pages, de la navigation, un formulaire et des appels API. Mais React seul a des limites : pas de SEO (le contenu est généré côté client), un premier chargement lent (tout le JS est téléchargé d'un coup), et un routing configuré manuellement.
Dans le **Lab 5**, vous migrerez l'annuaire vers **Next.js** : routing par fichiers, layouts, composants serveur, et `loading.js` automatique.

Exercice bonus

Bonus : page 404

Ajoutez une route « attrape-tout » pour les URL invalides :

```
<Route path="*" element={<NotFound />} />
```

Créez `src/pages/NotFound.jsx` avec un message « Page introuvable » et un lien vers l'accueil. Testez en tapant `/xyz` dans l'URL.

Lab 4 — Formulaires et React Router — Terminé ✓

Vous savez créer une application multi-pages avec
des routes, de la navigation, et des formulaires contrôlés.

Prochaine étape : Next.js et le rendu côté serveur.

Chaque page a son URL. Chaque URL a sa page.